

Generate–Verify–Repair for Intent-Driven Network Changes: Controlled Evaluation with a Deterministic Verifier

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We report a preregistered paired trial that measures whether a generate–verify–repair (GVR) pipeline—single shared LLM generation per intent, deterministic verification, and up to one automated repair—reduces accepted verifier-unsafe network changes relative to LLM-only generation on a standardized synthetic NetOps intent workload. Following preregistration (study-hosted-netops-gvr-v1-2026-08-29) we ran $N = 500$ distinct intents stratified across 10 synthetic topology profiles using gpt-5-mini for generation and a deterministic network verifier for adjudication. Observed accepted final changes: baseline (LLM-only) 496/500; guarded (GVR) 500/500. Paired contingency: $n_{11} = 496$, $n_{10} = 4$, $n_{01} = 0$, $n_{00} = 0$. The preregistered exact conditional two-sided McNemar test on the discordant margin ($n = 4$, $k = n_{10} = 4$) yields $p = 0.125$. A predeclared exploratory paired bootstrap percentile 95% CI (10,000 resamples; seed = 1729) for the paired difference = [0.002, 0.016]. Repairs were invoked for 4 initial candidates (one repair attempt per repaired task); total hosted model calls observed = 504 (500 shared_initial + 4 repair calls), mean model calls per accepted change = $504/500 = 1.008$. Because the preregistered decision rule is the exact conditional McNemar, we do not claim confirmatory statistical significance. We reconcile the conditional McNemar and bootstrap perspectives, document verifier semantics and known blind spots, clarify provider-call accounting semantics, present representative repaired-episode diagnostics, and point readers to the archived evidence bundle and preregistration for full machine-readable artifacts.

I. INTRODUCTION

Automating the translation from operator intent to device configuration can substantially reduce toil in network operations but introduces risk: incorrect or ill-sequenced CLI sequences can violate workflow, dependency, or availability constraints and produce outages. Large language models (LLMs) are increasingly used for intent translation and broader AIOps tasks; however, hallucination and factual errors remain primary reliability concerns for operational NetOps. A practical architectural mitigation is a generate–verify–repair (GVR) gate: produce an LLM candidate for a canonical CLI sequence, deterministically verify that candidate against explicit workflow, dependency, and availability rules, and, when the verifier flags violations, attempt a bounded automated repair. This paper presents a preregistered, artifact-grounded paired trial that asks whether a simple GVR pipeline (LLM $\rightarrow$ deterministic verifier $\rightarrow$ at most one automated repair per task) reduces verifier-unsafe accepted changes relative to accepting the shared LLM candidate (LLM-only baseline) on a standardized synthetic NetOps workload.

We executed the preregistered protocol (study-hosted-netops-gvr-v1-2026-08-29) on $N = 500$ paired intents stratified across 10 synthetic topology profiles. The execution used a single declared generator (gpt-5-mini) under a shared_initial_generation semantics (one initial model call per intent, shared across conditions) and a deterministic network verifier that enforces canonical parsing, sequencing and workflow policies, availability/group constraints, paired-field dependencies, and final-state equality to the task expected_final_state. Our contributions are: (1) a fully preregistered, reproducible paired trial with frozen seeds and archived artifacts; (2) an evidence-grounded description of generation/orchestration semantics, verifier capabilities, and the bounded repair loop; and (3) reconciled inferential reporting that follows the preregistered exact conditional McNemar decision rule while also presenting the predeclared exploratory bootstrap interval and per-episode verifier diagnostics for repaired cases. All numerical quantities below are reported verbatim from the archived execution and analysis artifacts.

II. RELATED WORK

Three strands of prior work motivated this study. First, surveys and systems papers document active interest and early deployments of LLMs for intent translation, AIOps, and multi-agent NetOps orchestration while repeatedly emphasizing hallucination and factual errors as obstacles to production automation [cited_record_ids]. Those works motivate adding deterministic validation gates into tool-augmented agentic pipelines. Second, methodological proposals for evaluate-and-repair architectures recommend standardized validator semantics, paired trial designs, frozen preregistration, and explicit accounting for orchestration costs (model calls, repair attempts) when assessing safety improvements; these methodological blueprints informed our preregistered design and choice of a paired primary estimator. Third, deterministic network configuration verifiers have matured to check syntax, sequencing/workflow patterns, availability constraints, and paired-field dependencies at scale; such verifiers detect many common LLM mistakes but differ in vendor and control-plane fidelity. Our verifier implements canonical parsing, required-sequence enforcement, group/availability constraints, paired-field dependency checks, and expected final-state comparison—sufficient to detect the four repaired episodes reported here but intentionally limited relative to

vendor or timing semantics. The present experiment situates itself as a controlled, reproducible evaluation of a compact GVR gate using those verifier capabilities rather than as a claim about production readiness across diverse vendor environments.

III. METHODOLOGY

Preregistration and design: We preregistered the full design (study-hosted-netops-gvr-v1-2026-08-29). The primary estimand is the paired per-intent success-rate difference (GVR - baseline) across $N = 500$ distinct intents stratified evenly across 10 synthetic topology profiles (50 intents per profile). Sampling seeds, the deterministic task generator, and the analysis executor were frozen before execution. The preregistered confirmatory test is the two-sided exact conditional McNemar on discordant pairs; a predeclared exploratory paired bootstrap percentile 95% CI (10,000 resamples; bootstrap_seed = 1729) was retained for descriptive estimation.

Execution semantics and orchestration: For each intent a single shared_initial_generation candidate was produced by the declared model (gpt-5-mini) under fixed generation settings. Under the guarded condition the GVR controller validated the shared candidate with the deterministic verifier and, if validation failed, issued at most one repair prompt to the same model. The baseline accepted the shared initial candidate without repair. The archived provider accounting records model_calls_used = 504 (500 shared_initial + 4 repair calls) and stage_counts showing shared_initial_generation = 500 and repair = 4; mean hosted model calls per accepted change equals $504/500 = 1.008$. For repaired pairs the observed model calls per pair = 2 (shared initial + one repair). The preregistered execution contract disallowed retrieval-augmented generation and limited maximum_repair_calls_per_task = 1.

Deterministic verifier: The adjudicating verifier canonicalizes CLI sequences and deterministically checks a rule set designed for the synthetic intent grammar. The pipeline performs (i) syntactic parsing and normalization; (ii) workflow and required-sequence checks (e.g., must_be_down_for_fields, all_down_before_any_field_change); (iii) availability/group constraints (exactly_active, minimum_active_in_groups); (iv) dependency and paired-field enforcement (paired MTU changes, equal_final_fields); and (v) final-state comparison to the task expected_final_state. Any nonempty violation set yields a failing outcome. For each episode the verifier emits normalized_configuration, observed_touched_field_count, parsed_command_count, required_command_count, a violation list and codes, and boolean valid. A human-readable codebook maps violation codes to descriptions. The verifier intentionally omits vendor-specific CLI semantics, control-plane timing/convergence modeling, session/execution-time errors, and hardware resource constraints; therefore verifier PASS is necessary for experimental safety but not sufficient for production safety.

Repair policy and prompt semantics: When the verifier fails the shared_initial candidate the GVR controller constructs

a constrained repair prompt containing: the original intent abstraction, the verifier violation codes with short descriptions, and an explicit instruction to produce a corrected canonical CLI sequence. At most one automated repair attempt was permitted per failing episode; the repair candidate was accepted only if the verifier returned valid = true. All repair prompts, repair responses, and verifier traces for repaired episodes are archived.

IV. RESULTS

Execution completeness and primary counts: The run completed every planned episode (completed_episode_count = 1000; completed_pair_count = 500) with no failed episodes. Observed accepted-change counts were baseline 496/500 (0.992) and guarded 500/500 (1.000). Paired contingency counts were $n_{11} = 496$, $n_{10} = 4$, $n_{01} = 0$, $n_{00} = 0$. Repairs were invoked in four episodes and each repair attempt produced a subsequent verifier PASS. Provider accounting: model_calls_used = 504 (shared_initial_generation = 500; repair = 4).

Preregistered inferential analysis: The preregistered confirmatory test conditions on the observed discordant margin $n = n_{10} + n_{01}$ and evaluates a two-sided binomial tail on $k = n_{10}$ with $p = 0.5$ (exact conditional McNemar). For $n = 4$ and $k = 4$ the one-sided tail $P(X \geq 4; n = 4, p = 0.5) = (1/2)^4 = 1/16$; doubling for two-sided yields $p = 0.125$. Under the preregistered decision rule this p-value does not meet the confirmatory threshold and we therefore do not claim confirmatory statistical significance.

Predeclared exploratory interval estimate: The predeclared paired bootstrap percentile 95% CI (10,000 resamples; seed = 1729) for the paired difference is [0.002, 0.016]. This bootstrap resampling of paired outcomes does not condition on the observed discordant margin; with only four discordants the percentile interval can exclude zero when resamples repeatedly include the few favorable discordant outcomes. Because the bootstrap interval was predeclared exploratory we report it descriptively and treat the McNemar result as the primary confirmatory outcome.

Repaired-episode diagnostics (artifact-grounded): The four repaired/discordant pairs corresponded to verifier-detectable sequencing or group/paired-field dependency violations. The archived verifier violation codes and short descriptions (as recorded) are: - pair-000090 | violation_codes: [MUST_BE_DOWN_FOR_FIELD] — "VLAN/field changed while interface remained admin up; required 'must be down' sequencing violated." | repair_invoked: yes | model_calls_used_for_pair: 2 | final_score: PASS. - pair-000341 | violation_codes: [MAKE_BEFORE_BREAK] — "Backup uplink not enabled before primary was shut; violates 'make-before-break' availability rule." | repair_invoked: yes | model_calls_used_for_pair: 2 | final_score: PASS. - pair-000381 | violation_codes: [PAIRED_MTU_CHANGE] — "MTU changed on one side of a paired link without coordinating the paired interface's MTU change (paired-field dependency)."

```

| repair_invoked: yes | model_calls_used_for_pair: 2
| final_score: PASS. - pair-000413 | violation_codes:
[GROUP_AVAILABILITY_CONSTRAINT] — "Both edge
interfaces placed admin down simultaneously, violating
minimum_active_in_groups availability constraint." |
repair_invoked: yes | model_calls_used_for_pair: 2 |
final_score: PASS. Representative per-episode verifier
diagnostics, normalized configurations, violation lists,
parsed/required command counts, and repair prompt/response
transcripts for the repaired episodes are archived in the
evidence bundle. The numerical totals and contingency counts
above are taken directly from the archived analysis.

```

V. DISCUSSION

Reconciling McNemar and bootstrap perspectives: The GVR pipeline intercepted four verifier-detectable unsafe initial candidates that baseline would have accepted; these four correspond to the observed discordants ($n = 4$, $k = 4$). The preregistered exact conditional McNemar conditions on the observed discordant margin and computes a binomial tail; with $n = 4$ this yields two-sided $p = 0.125$ and thus does not meet the preregistered confirmatory threshold. The predeclared exploratory bootstrap percentile CI excludes zero ($[0.002, 0.016]$) because bootstrap resampling does not condition on margins and may repeatedly sample the same small set of favorable discordant outcomes. Both perspectives are informative: the exact conditional McNemar enforces the preregistered confirmatory decision rule and is conservative for small discordant counts, while the bootstrap interval provides a descriptive estimate of effect magnitude but suffers reduced frequentist coverage guarantees when discordant counts are small. We adhere to the preregistered McNemar decision rule for the primary claim and present the bootstrap interval as a predeclared exploratory descriptor.

Verifier scope and operational sufficiency: The deterministic verifier enforces canonical normalization, required sequencing, availability/group constraints, dependency and paired-field rules, and final-state equality to the task expected_final_state. These checks are necessary to detect many syntactic and workflow errors that LLMs commonly produce within our intent grammar and were sufficient to flag the four repaired episodes in this run. Critically, the verifier does not model vendor-specific CLI semantics, control-plane timing and convergence, session execution errors, or hardware resource limits; therefore a verifier PASS is necessary but not sufficient to guarantee production safety. Production automation would require expanded verifier fidelity (vendor parsers, control-plane simulators, timing/convergence models) and live telemetry-driven validation (pre- and post-change) before unsupervised execution.

Operational cost and scaling: Under the bounded repair policy (maximum_repair_calls_per_task = 1) the repair invocation rate was 4/500 (0.8%), yielding modest additional model calls: hosted model call events = 504 (500 shared initial + 4 repairs). This low overhead in the synthetic workload indicates that a compact deterministic verifier can act as a low-latency

gate and remove verifier-detectable unsafe proposals at modest model-call cost when baseline failure rates are low. Realized benefits will depend on baseline failure rates, verifier runtime at inventory scale, adversarial or more diverse intent distributions, and latency tradeoffs of higher-fidelity verifiers.

Threats to validity and generality: Internal validity is strengthened by preregistration, frozen seeds, deterministic task generation, and complete execution accounting (complete_pair_count = 500; failed_episode_count = 0). Construct validity is limited by the verifier abstraction level and the synthetic device templates: the verifier enforces workflow and group constraints matching the synthetic intent grammar, but these rules imperfectly model vendor behavior and dynamic control-plane hazards. External validity is constrained by single-model scope (gpt-5-mini) and synthetic topologies; multi-model replication, vendor-specific parsing, and live telemetry tests are required to generalize. Conclusion validity was affected by realized margins: the preregistered power plan targeted an absolute paired increase ≈ 0.08 ; the observed near-ceiling baseline ($496/500 = 0.992$) produced only four discordant pairs and reduced realized power relative to the preregistered assumption. These limitations explain why a modest numerical paired difference appears in the exploratory bootstrap interval but not under the preregistered exact McNemar decision rule.

Practical implications: For organizations adopting LLM-driven intent translation, a compact deterministic verifier used as an automated gate plus a bounded automated repair loop can intercept many verifier-detectable unsafe proposals with low model-call overhead in workloads similar to our synthetic setting. However, engineering decisions—verifier fidelity, permitted repair depth, model selection, and latency budget—must be calibrated against operational risk tolerances, expected baseline error rates, and adversarial threat models. The preregistered templated-automation comparator (secondary confirmatory arm) was not executed in this run; therefore multiplicity adjustments planned in the preregistration were not applied to the present confirmatory test.

VI. LIMITATIONS

- Synthetic tasks and topology profiles were used (synthetic_topologies_v1 and synthetic_device_configs_v1); external validity to production hardware, vendor-specific CLIs, live telemetry, and control-plane timing effects is untested.
- Only a single generation model (gpt-5-mini) was evaluated; multi-model generality and replication remain to be established.
- Safety in this experiment is defined relative to the deterministic verifier’s encoded rule set (syntactic parsing/normalization, required-sequence/workflow-policy checks, protected-interface rules, group and availability constraints, dependency-rule enforcement, and final-state comparison to the task expected_final_state). Passing

this verifier is necessary for the experiment’s outcome but not sufficient for all real-world safety properties.

- The preregistered power plan targeted an absolute paired increase ≈ 0.08 (8 percentage points). The observed baseline success rate ($496/500 = 0.992$) produced only four discordant pairs, substantially reducing realized power relative to the preregistered assumption and limiting confirmatory sensitivity.
- The preregistered templated-automation comparator was not executed; therefore the preregistered multiplicity plan could not be fully applied and the familywise error interpretation across two planned tests is not applicable to the present execution.

VII. CONCLUSION

In a preregistered paired trial ($N = 500$) using gpt-5-mini and a deterministic verifier, a generate–verify–repair pipeline repaired four verifier-detectable unsafe initial candidates at modest model-call cost (4 repairs; `model_calls_used` = 504). The paired difference = 0.008 produces an exploratory bootstrap 95% CI [0.002, 0.016] while the preregistered exact conditional McNemar test ($n_{10} = 4$, $n_{01} = 0$) yields $p = 0.125$; therefore we do not claim confirmatory statistical significance under the preregistered decision rule. Deterministic verification combined with a bounded repair loop can remove verifier-detectable unsafe outputs with low overhead in the studied synthetic setting; broader operational claims require expanded verifier fidelity, adversarial NetOps evaluation, multi-model replication, and execution of the preregistered templated-automation comparator which was not executed in this run. Archival artifacts including the preregistration, model configuration, raw results, per-episode verifier traces, provider accounting, and analysis logs are available in the evidence bundle for independent verification and audit (see the preregistration record).

DISCLOSURE STATEMENT

This study was executed autonomously after an explicit autonomy lock. The generation model used in the archived run was gpt-5-mini (`declared_model_version` recorded in the archived model configuration). Key automated components in the run were a deterministic task generator, a deterministic network verifier, and a GVR controller implementing `shared_initial_generation` plus at most one automated repair per task. The study was preregistered under study-hosted-netops-gvr-v1-2026-08-29. Primary high-level execution quantities reported here are: $N = 500$ complete pairs; `model_calls_used` = 504; repair calls = 4; paired counts $n_{11} = 496$, $n_{10} = 4$, $n_{01} = 0$, $n_{00} = 0$. The immutable initial master prompt for this run is archived (`provenance/master_prompt.txt`) and its SHA-256 is 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba. The design, preregistration, code generation, execution, analysis, manuscript drafting, autonomous peer review, and this revision were performed autonomously after the autonomy lock; no human manually edited or rewrote manuscript technical content after

that lock. The preregistered templated-automation comparator (secondary confirmatory arm) was not executed in this archived run; therefore the original preregistered multiplicity plan that would have adjusted for two confirmatory tests was not applied.

REFERENCES

- [1] doi:10.1145/3718958.3750537
- [2] Sebastian Simon, Alina Mailach, Johannes Dorn, Norbert Siegmund, "A Methodology for Evaluating RAG Systems: A Case Study On Configuration Dependency Validation", 2024, 10.48550/arxiv.2410.08801
- [3] Nguyen Tu, Sukhyun Nam, James Won-Ki Hong, "Intent-Based Network Configuration Using Large Language Models", 2024, 10.1002/nem.2313
- [4] Lingzhe Zhang, Tong Jia, Mengxi Jia, Yifan Wu, Aiwei Liu, Yong Yang, Zhonghai Wu, Xuming Hu, Philip S. Yu, Ying Li, "A Survey of AIOps in the Era of Large Language Models", 2025, 10.1145/3746635
- [5] Zhaodong Wang, Samuel Lin, Guanqing Yan, Soudeh Ghorbani, Minlan Yu, Jiawei Zhou, Nathan Hu, Lopa Baruah, Sam Peters, Srikanth Kamath, Jian Yang, Ying Zhang, "Intent-Driven Network Management with Multi-Agent LLMs: The Confucius Framework", 2025, 10.1145/3718958.3750537